of the host system. To attach the Docker port to an external system port, use *–p*:

```
#docker run -it -v /temp/:/home/ -p 4567:80 kalilinux/kali-linux-docker /bin/bash
```

This will attach the external port 4567 to the container's port 80. This can be very useful for SaaS and PaaS, provided that the deployed application needs to connect to the external network. Running GUI applications on Docker can often be another requirement. Docker doesn't have x server defined so, to do that, you need to mount the x server file to the Docker instance.

```
#docker run -it -v -e DISPLAY=$DISPLAY -v /tmp/.X11-unix:/tmp/.X11-unix \ kalilinux/kali-linux-docker /bin/bash
```

This will forward the X11 socket to the container inside Docker. To ship the Docker image to another system, you need to push it on the Docker online repository, as follows:

```
#docker push new_image
```

You can even save the container image in the tar archive:

```
#docker export new_image
```

There is a lot more to learn on Docker, but going deeper into the subject is not the purpose of this article. The positive point about Docker is its many tutorials and hacks available online, from which you can easily get a better understanding of how to use it to get your work done. Since its first release in 2013, Docker has improved so much that it can be deployed in a production or testing environment because it is easy to use.

There are other solutions made for Docker, which are designed for all scenarios. These include Kubernetes (a Google project for the orchestration of Docker), Swarm and many more services for Docker migrations, which provide graphical dashboards, etc. Automation tools for systems admins like Puppet and Chef are also starting to provide support to Docker containers. Even systemd has started to provide a management utility for nspawn and other containers with a number of tools like machinectl and journalctl.

machinectl

This comes pre-installed with the systemd init manager. It is used to manage and control the state of the systemd based virtual machine, and the container works underneath the systemd service. To see all containers running in your system, use the command given below:

```
#machinectl -a
```

To get a status of any running container, use the command given below:

```
#machinectl status my_deb
```

Note: machinectl doesn't show Docker containers, since the latter run behind the Docker daemon.

To log in to a container, use the command given below:

```
#machinectl login my_deb
```

Switch off a container, as follows:

```
#machinectl poweroff my_deb
```

To kill a container forcefully, use the following command:

```
#machinectl -s kill my_deb
```

To view the logs of a container, you can use journalctl, as follows:

```
#journalctl -M my_deb
```

What to get from this article

Sandboxes are important for every IT professional, but different professionals may require different solutions. If you are a developer or application tester, *chroot* may not be a good solution as it allows attackers to escape from the *chroot* jail. Weak containers like systemd-nspawn or firejail can be a good solution because they are easy to deploy. Using Docker-like containers for application testing can be a minor headache, as making your container ready for your process to run smoothly can be a little painful.

If you are a SaaS or PaaS provider, containers will always be the best solution for you because of their strong isolation, easy shipping, live migration and clustering-like features. You may go with traditional virtualisation solutions (virtual machines), but resource management and quick booting can only be got with containers. END

References

[1] Namespaces: *http://lwn.net/Articles/531114/*
[2] Chroot: *https://help.ubuntu.com/community/BasicChroot*
[3] Firejail: *https://l3net.wordpress.com/2014/09/19/*

By: Shubham Dubey

The author is a Linux enthusiast. His work and development lies in the field of cloud computing, virtualisation and cyber security. You can contact him at *sdubey504@gmail.com*.